第 27 章：用 pytest 验证异步代码与故障路径

先记住一句话

异步测试的核心不是“给 `async def` 加一个标记”，而是把任务何时开始、何时失败、谁负责取消、何时完成清理，都变成可以观察和断言的测试合同。

正常路径通常很好演示：模型返回答案，多个工具都成功，流被完整消费。真正容易在生产环境出问题的，往往是另外一半：

- 坏参数有没有在 I/O 前被拒绝；
- 一个并发任务失败后，其他任务是否被取消并等待完毕；
- timeout 到期后，后台是否还留着 Task；
- 消费者提前离开流时，连接是否已经归还；
- 两个请求交错运行时，`ContextVar` 是否串值；
- 测试替身究竟验证了真实协议的哪一层。

本章把这些问题写成可重复、离线、失败时能准确指出原因的 pytest 测试。学完后，你应该能：

- 使用 pytest 的发现规则、原生 `assert`、参数化和 `raises`；
- 用 fixture 表达资源的创建、借用、作用域和清理；
- 让 pytest-asyncio 管理测试事件循环，避免生命周期错配；
- 正确选择 Fake、`Mock`、`AsyncMock`、`monkeypatch` 和进程内 transport；
- 不依赖随机 `sleep`，稳定验证取消、TaskGroup、并发上限和上下文隔离；
- 检查 `ExceptionGroup` 的真正异常叶子，而不是只验证外层容器；
- 清楚说明一条离线测试证明了什么，以及它没有证明什么。

1. Java → Python 心智模型

Java 中熟悉的做法	Python / pytest 中可以先想到	最重要的不等价之处
JUnit 5 的 `@Test`	名字以 `test_` 开头的函数	pytest 默认靠命名发现，测试不必放进类中
JUnit assertions / AssertJ	Python 原生 `assert`	pytest 会重写测试模块中的断言，失败时展示表达式中的值

1. Java → Python 心智模型

Java 中熟悉的做法	Python / pytest 中可以先想到	最重要的不等价之处
JUnit 5 的 `@Test`	名字以 `test_` 开头的函数	pytest 默认靠命名发现，测试不必放进类中
JUnit assertions / AssertJ	Python 原生 `assert`	pytest 会重写测试模块中的断言， 失败时展示表达式中的值
`@ParameterizedTest` + `@MethodSource`	`@pytest.mark.parametrize`	每组参数在收集阶段变成独立测试项， 不是测试函数内部的一次循环
`assertThrows`	`pytest.raises`	上下文范围应只包真正期望抛错的调用
`@BeforeEach` / `@AfterEach`	function-scope fixture	fixture 还能依赖其他 fixture， 并通过函数参数按名字注入
`try-with-resources`	`yield` fixture / `async with`	`yield` 前是 setup，`yield` 后是 teardown； 资源所有者仍是 fixture
JUnit `@TempDir`	pytest 内置 `tmp_path`	得到的是每个测试独立的 `pathlib.Path`
Mockito `Mock`	`unittest.mock.Mock`	普通 Mock 的调用结果不可自动 `await`
对异步接口做 Mockito stub	`AsyncMock` 或 async Fake	`called` 和 `awaited` 是两种不同事实
替换静态调用或全局依赖	`monkeypatch`	要替换被测模块查找名字的位置， 而不一定是原始定义位置
`CountDownLatch` / `CyclicBarrier`	`asyncio.Event` / `asyncio.Barrier`	它们协调的是 coroutine Task， 不会阻塞事件循环线程
`CompletableFuture` 测试	创建 Task 后 `await`	`Task.cancel()` 是协作式取消请求， 不是强制终止线程
Java 结构化并发作用域	`asyncio.TaskGroup`	多个失败通过 `ExceptionGroup` / `BaseExceptionGroup` 传播

pytest fixture 可以先类比为“JUnit 生命周期钩子加参数注入”，但不能完全等同于 Spring 容器。
pytest 根据测试函数的参数名构建依赖图，按需创建并缓存值；它不会扫描业务 Bean，也不会提供完整的应用依赖注入生命周期。

pytest-asyncio 则可以先类比为“JUnit 的异步测试执行扩展”，它给 coroutine 测试提供正在运行的

README README README
'lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

pytest-asyncio 则可以先类比为"JUnit 的异步测试执行扩展"。它给 coroutine 测试提供正在运行的事件循环，但不会替业务代码决定 Task、客户端和流由谁关闭。

2. pytest 怎样找到并执行一条测试

本章的 `pytest.ini` 把发现范围明确为：

```
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
```

不传文件路径时，pytest 从 `tests/` 递归查找 `test_*.py`，再收集模块顶层的 `test_*` 函数，以及 `Test*` 类中的 `test_*` 方法。测试类不是必需的，也不应该为了分组而承担共享可变状态。

最小测试只是一个普通函数：

```
def test_model_accepts_valid_search_args() -> None:
    args = SearchArgs.model_validate({"query": "  asyncio  ", "limit": 2})

    assert args.query == "asyncio"
    assert args.limit == 2
```

执行顺序是：pytest 收集函数，调用函数，Pydantic 校验并规范化输入，两个 `assert` 检查公开结果，函数正常返回后测试通过。

Python 的 `assert` 原本只会在条件为假时抛 `AssertionError`。pytest 在导入测试模块时会重写断言，因此 `assert actual == expected` 失败时通常能展开左右值、集合差异或第一个不同位置。一条断言仍应表达一条清晰合同，不要把多个无关结论塞进一个巨大布尔表达式。

不要用 `python -O -m pytest` 运行测试。优化模式会移除未经过 pytest 重写的普通 `assert`，例如业务模块和插件中的断言，可能让测试失去本应检查的条件。

精确运行一个测试节点

从本章目录可以只运行一个"文件 :: 测试函数"节点：

精确运行一个测试节点

从本章目录可以只运行一个"文件 :: 测试函数"节点：

```
../../../.venv/bin/python -m pytest \
  tests/test_async_runtime.py::test_cancellation_waits_for_start_then_checks_finally -vv
```

这相当于在 JUnit 中只运行一个测试方法，适合定位失败和快速迭代。

3. 用参数化和 `raises` 描述输入合同

同一条规则对应多组输入时，不要在测试函数里手写循环。参数化会让每组输入成为独立测试项：

```
@pytest.mark.parametrize(
    ("raw_args", "bad_field"),
    [
        pytest.param({"query": "", "limit": 2}, "query", id="empty-query"),
        pytest.param({"query": "ok", "limit": 0}, "limit", id="limit-too-small"),
        pytest.param({"query": "ok", "limit": "2"}, "limit", id="strict-integer"),
    ],
)
def test_model_rejects_bad_search_args(raw_args: object, bad_field: str) -> None:
    with pytest.raises(ValidationError) as raised:
        SearchArgs.model_validate(raw_args)

    locations = {str(error["loc"][0]) for error in raised.value.errors()}
    assert bad_field in locations
```

这里有三个重要行为：

1. pytest 在收集阶段生成三个测试项，并在报告中使用自定义 `id`；
2. 一项失败不会遮住另外两项的结果；
3. 参数值不会被自动深拷贝，共享并修改同一个 list 或 dict 仍可能污染后续测试项。

`pytest.raises(ExpectedError)` 只证明上下文中的代码抛出了目标类型或其子类。因此作用域要尽量窄，并继续检查异常的关键字段、消息或精确类型：

```
# 正确：只有预期失败的操作位于 raises 中。
```

README README README
h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

```
# 正确：只有预期失败的操作位于 raises 中。
with pytest.raises(ValidationError) as raised:
    SearchArgs.model_validate(raw_args)
assert "query" in str(raised.value)

# 反例：准备代码碰巧抛出同类异常时，测试也可能假通过。
with pytest.raises(ValidationError):
    raw_args = complicated_setup()
    SearchArgs.model_validate(raw_args)
```

Java 中的 `assertThrows(RuntimeException.class, ...)` 也会接受子类；这里同理。合同若要求精确类型，可以继续断言 `raised.type is ValidationError`。

4. fixture：测试借用资源，fixture 拥有资源

fixture 是 pytest 按测试参数名解析的依赖。例如：

```
@pytest.fixture
def isolated_events() -> list[str]:
    return []


def test_starts_empty(isolated_events: list[str]) -> None:
    assert isolated_events == []
```

pytest 发现 `isolated_events` 参数后，先执行同名 fixture，再把返回值传给测试。fixture 默认是 `function` 作用域，所以每个测试得到新的 list，不依赖执行顺序。

作用域决定缓存多久

scope	复用范围	适合的资源	主要风险
`function`	每个测试项一次	可变数据、客户端、临时业务资源	创建次数较多，但隔离最好
`class`	一个测试类	类内共享的只读配置	类内测试可能互相污染
`module`	一个测试模块	昂贵且只读的模块资源	模块内出现顺序依赖
`package`	一个测试包	包级只读资源	所有权不容易看见

作用域决定缓存多久

scope	复用范围	适合的资源	主要风险
`function`	每个测试项一次	可变数据、客户端、临时业务资源	创建次数较多，但隔离最好
`class`	一个测试类	类内共享的只读配置	类内测试可能互相污染
`module`	一个测试模块	昂贵且只读的模块资源	模块内出现顺序依赖
`package`	一个测试包	包级只读资源	所有权不容易看见
`session`	整次 pytest 进程	极昂贵且不可变的环境元数据	一次修改污染大量测试

不要只为了减少创建次数，就把可变集合、事务中的数据库连接或绑定事件循环的客户端提升为 `session`。昂贵资源确实必须共享时，共享层尽量只读，每个测试会修改的数据仍保持 function scope，或者为每项提供明确的回滚与重建。

`yield` 把资源生命周期分成两半

```
@pytest.fixture
def note_database(tmp_path: Path) -> Iterator[Path]:
    path = tmp_path / "agent-notes.sqlite3"
    initialize_notes(path)
    try:
        yield path
    finally:
        path.unlink(missing_ok=True)
```

执行顺序是：

```
fixture setup → yield path → 测试使用 path → 回到 fixture → finally 清理
```

测试成功、断言失败或抛出异常，pytest 都会继续执行已经到达 `yield` 的 fixture teardown。多个 fixture 按依赖顺序创建，并对已成功创建的部分按反方向清理。这与嵌套 `try-with-resources` 的“后创建先关闭”很像。

但如果异常发生在某个 fixture 到达 `yield` 之前，它自己的 `yield` 后代码不会执行。setup 每取得一项资源，就要尽快进入相应的上下文管理器或注册安全清理，不能等所有 setup 完成后才统一补救。

5. pytest-asyncio：事件循环由谁拥有

pytest 本身不会等待 coroutine 测试。本章安装 pytest-asyncio，并在 strict 模式下显式声明异步测试和异步 fixture：

```
asyncio_mode = strict
asyncio_default_fixture_loop_scope = function
asyncio_default_test_loop_scope = function
```

```
@pytest.mark.asyncio
async def test_async_behavior() -> None:
    assert asyncio.get_running_loop().is_running()


@pytest_asyncio.fixture
async def model_client() -> AsyncIterator[httpx.AsyncClient]:
    def handler(request: httpx.Request) -> httpx.Response:
        return httpx.Response(200, json={"answer": "离线结果"})

    transport = httpx.MockTransport(handler)
    async with httpx.AsyncClient(
        transport=transport,
        base_url="https://model.test",
    ) as client:
        yield client
```

strict 模式下：

- coroutine 测试需要 `@pytest.mark.asyncio`；
- async fixture 需要 `@pytest_asyncio.fixture`；
- 本章每个测试函数使用独立事件循环；
- pytest-asyncio 在该循环中把测试作为 Task 运行，所以测试体直接 `await`。

为什么不能再调用 `asyncio.run()`

`asyncio.run()` 是同步程序最外层的 runner：它创建事件循环，执行一个异步入口，最后关闭循环。
pytest-asyncio 已经在当前线程启动了循环，因此下面的写法会失败：

```
@pytest.mark.asyncio
```

README README README
7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
2.60
需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

为什么不能再调用 `asyncio.run()`

`asyncio.run()` 是同步程序最外层的 runner：它创建事件循环，执行一个异步入口，最后关闭循环。pytest-asyncio 已经在当前线程启动了循环，因此下面的写法会失败：

```
@pytest.mark.asyncio
async def test_wrong() -> None:
    result = asyncio.run(call_tool())  # RuntimeError: 不能嵌套运行事件循环
```

测试中应直接写：

```
@pytest.mark.asyncio
async def test_right() -> None:
    result = await call_tool()
```

本章的普通脚本 `example.py` 没有外部 runner，所以只在最外层调用一次 `asyncio.run()`；测试入口则完全交给 pytest-asyncio。这是两个不同入口的所有权边界。

fixture scope 与 loop scope 是两条维度

fixture 的 `scope` 决定 fixture 值缓存多久，`loop_scope` 决定异步 setup/teardown 在哪个事件循环执行。异步 fixture 的 loop scope 不能比缓存 scope 更窄，否则一个仍在复用的资源可能所属循环已经关闭。

本章不共享异步资源，所以测试与 fixture 都采用 function loop scope。若确实要跨测试共享客户端，需要同时设计缓存作用域、事件循环作用域和可变状态隔离，不能只把 `scope="module"` 加到一个地方。

6. Fake、Mock、AsyncMock 与 monkeypatch 怎样选

测试替身的目标是控制边界，而不是把所有对象都变成 Mock。

手段	最适合回答的问题	不适合证明的事情
普通函数 / 小 Fake	输入怎样映射为结果，或对象怎样经历多个状态	真实网络、真实数据库、生产性能
`Mock`	同步依赖被调用几次、收到什么参数	async 调用是否真的被等待
`AsyncMock`	async 依赖被 await 几次、收到什么参数	完整的并发或资源状态机
`monkeypatch`	一个测试期间临时替换属性、环境变量或字典项	跨进程依赖或任意位置的全局替换

README README README
7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

手段	最适合回答的问题	不适合证明的事情
普通函数 / 小 Fake	输入怎样映射为结果，或对象怎样经历多个状态	真实网络、真实数据库、生产性能
`Mock`	同步依赖被调用几次、收到什么参数	async 调用是否真的被等待
`AsyncMock`	async 依赖被 await 几次、收到什么参数	完整的并发或资源状态机
`monkeypatch`	一个测试期间临时替换属性、环境变量或字典项	跨进程依赖或任意位置的全局替换
`MockTransport`	HTTPX 怎样构造请求、处理状态和解析响应	DNS、socket、TLS、代理和真实连接池

普通 Mock 不能冒充 async 调用

```
plain_mock = Mock(return_value="普通字符串")
await plain_mock("prompt")  # TypeError: str 不可等待
```

调用 `AsyncMock` 才会先得到 awaitable；等待该对象后，才得到配置的 `return_value`：

```
model_call = AsyncMock(return_value="离线模型结果")

result = await ask_model(model_call, "解释 TaskGroup")

assert result == "离线模型结果"
model_call.assert_awaited_once_with("解释 TaskGroup")
```

`model_call.called` 只说明发生过函数调用，不说明返回的 coroutine 被等待。异步合同应使用 `assert_awaited*` 系列断言。

monkeypatch 要替换“使用位置”

假设被测模块执行：

```
from gateway import call_model_api
```

导入完成后，被测模块已经拥有自己的 `call_model_api` 名字。只修改 `gateway.call_model_api` 不一定能影响它；应该 patch 被测模块实际查找的属性：

```
fake_api = AsyncMock(return_value="patched-result")
```

README README README
7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

```
fake_api = AsyncMock(return_value="patched-result")
monkeypatch.setattr(agent_runtime, "call_model_api", fake_api)
```

pytest 会在当前测试结束后撤销 monkeypatch。这个机制类似局部的名字重绑定，不是 Java 动态代理，也不会自动替换其他模块已经绑定的同名对象。

选择顺序

1. 只需要固定输入输出时，优先传普通函数或很小的 Fake；
2. 需要检查同步调用的次数和参数时，用 `Mock`；
3. 接口是 async 时，用 `AsyncMock` 并断言 await；
4. 依赖尚未通过参数注入时，用 monkeypatch 替换被测模块的使用位置；
5. 替身开始复制复杂生产协议时，增加更真实的一层测试，而不是继续堆 Mock 配置。

7. 稳定异步测试：同步状态，不猜时间

异步测试最通用的结构是：

```
安排任务
  → 等待被测对象进入确定状态
  → 触发成功、失败、取消或放行
  → await 所有者直到任务到达终态
  → 断言结果、异常、兄弟任务和资源清理
```

关键是“进入确定状态”。`await asyncio.sleep(0.01)` 只能猜目标 Task 大概运行到哪里；CI 负载、事件循环调度和机器速度都可能改变结果。`Event`、`Barrier`、队列或可观察调用记录能表达真实的先后关系。

取消不是同步终止

稳定的取消测试可以写成：

```
task = asyncio.create_task(wait_for_tool(lifecycle))
await lifecycle.started.wait()  # 已确认进入资源区

task.cancel("测试主动取消")
with pytest.raises(asyncio.CancelledError):
```

README README README
s/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

取消不是同步终止

稳定的取消测试可以写成：

```
task = asyncio.create_task(wait_for_tool(lifecycle))
await lifecycle.started.wait()  # 已确认进入资源区

task.cancel("测试主动取消")
with pytest.raises(asyncio.CancelledError):
    await task

assert lifecycle.cleaned.is_set()
assert lifecycle.events == ["started", "cleaned"]
assert task.cancelled()
```

`task.cancel()` 只是安排在 Task 的下一次可取消点抛出 `CancelledError`。只有继续 `await task`，调用方才能确认异常已传播、`finally` 已执行、Task 已到达终态。只调用 `cancel()` 就结束测试，可能留下 pending Task，也没有证明连接或锁已经释放。

测试 timeout 是保险丝，不是业务实现

等待阶段可以加一个较宽的测试超时：

```
async with asyncio.timeout(1.0):
    await lifecycle.started.wait()
```

它防止实现退化后整套 CI 永久挂住，但不能替代业务合同：

- 它不能证明被测服务使用了正确 deadline；
- 它不能替被测代码执行 `finally`；
- 它不能"硬杀"吞掉取消的代码；
- 清理本身需要等待时，墙钟耗时可能超过触发 timeout 的那个时刻。

8. TaskGroup：先决定失败语义，再写断言

并发工具不是一律"一个失败全部失败"。要先区分业务合同。

可选任务：只把预期业务失败变成数据

8. TaskGroup：先决定失败语义，再写断言

并发工具不是一律"一个失败全部失败"。要先区分业务合同。

可选任务：只把预期业务失败变成数据

```python
async def as_optional_outcome(name: str, call: ToolCall) -> ToolOutcome:
    try:
        return ToolOutcome(name=name, value=await call(), error=None)
    except ToolBusinessError as error:
        return ToolOutcome(name=name, value=None, error=str(error))
```

这里只有 `ToolBusinessError` 属于允许的部分结果。`ToolProgramError`、`TypeError` 和取消不能被 `except Exception` 粗暴吞掉，否则程序缺陷会伪装成业务失败。

相应测试应同时断言：成功项仍存在，已知失败项保留正确错误，所有成员都完成清理。不要只断言最终列表长度。

必需任务：一个失败，兄弟也必须被收集

全部工具都是必需项时，成员异常离开 coroutine，`TaskGroup` 会取消尚未结束的兄弟，并等待它们退出，然后把未处理错误组合后抛给父任务。

一个稳定测试应按以下顺序控制：

1. 用 Event 等待三个 Fake 都进入 `started`；
2. 只放行会抛 `ToolProgramError` 的成员；
3. 保证另外两个兄弟仍停在可取消等待点；
4. `await` 父请求并取得 `ExceptionGroup`；
5. 检查真实异常叶子、兄弟确实收到取消、所有 `finally` 都已完成。

测试不应断言兄弟的启动或清理先后顺序。Task 调度顺序通常不是业务合同；"全部已清理"才是。

deadline 要覆盖整个并发作用域

本章用一次绝对截止时间包住整个任务组：

```python
async with asyncio.timeout_at(deadline):
    return await run_required_tools(tools)
```

README README README
lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

deadline 要覆盖整个并发作用域

本章用一次绝对截止时间包住整个任务组：

```
async with asyncio.timeout_at(deadline):
    return await run_required_tools(tools)
```

期限到达后，当前父 Task 收到取消，TaskGroup 再取消并等待子任务；离开 timeout 上下文后，调用方看到 `TimeoutError`。测试要验证的是：

- 调用方收到 `TimeoutError`；
- 未完成工具观察到取消；
- 工具的异步清理在返回前完成。

不要断言“必须恰好在 0.200 秒返回”。deadline 决定何时发起取消，资源清理仍需要时间。

9. ExceptionGroup：检查叶子，而不是外壳

TaskGroup 可能报告嵌套异常组。下面的断言过宽：

```
with pytest.raises(ExceptionGroup):
    await request()
```

它只证明出现了异常组。组里即使是意外的 `TypeError`，测试也会通过。更可靠的做法是取得异常组，递归展开叶子，再按业务语义检查：

异常叶子	建议分类	默认处理方向
`ToolBusinessError`	预期业务失败	只有合同允许时才转成部分结果
`TimeoutError`	deadline / timeout	记录预算层次，不伪装成业务拒绝
`CancelledError`	上游取消	本层清理后继续传播
其他异常	程序缺陷	保留 traceback，让请求和测试失败

`CancelledError` 继承 `BaseException`。包含它的手工异常组必须是 `BaseExceptionGroup`，不一定是普通 `ExceptionGroup`。TaskGroup 因成员失败而取消兄弟时，兄弟正常传播的 `CancelledError` 通常不会成为最终错误叶子，所以还要通过 Fake 状态单独证明兄弟确实被取消并清理。

README README README
2x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

pytest 9 也提供 `pytest.RaisesGroup` 描述异常组形状。本章显式展开叶子，是为了让"如何分类失败"这个过程更容易观察；只关心固定组结构时可以进一步使用 `RaisesGroup` 。

10. 三类容易写成随机测试的并发合同

并发上限：测状态，不测速度

"六项工作在 300 毫秒内完成"不能证明在途数从未超过 2。更稳妥的 Fake 在 Semaphore 保护区域记录状态：

```
self.active += 1
self.max_active = max(self.max_active, self.active)
if self.active == self.expected_peak:
    self.peak_reached.set()
try:
    await self.release.wait()
finally:
    self.active -= 1
```

测试等 `peak_reached` 后，在闸门仍关闭时断言 `active == 2`，再放行全部工作。结束后检查：

- `max_active == 2`：确实用满两个槽位，也从未超过上限；
- 返回结果与输入的对应关系正确；
- `active == 0`：所有 `finally` 都归还了探针状态。

Semaphore 验证的是同时在途数，不是单位时间请求数。不要把并发上限测试写成 QPS 限流测试。

提前退出异步流：返回前就应完成关闭

`async for` 提前 `return` 或 `break` 时，资源所有者应该显式关闭异步生成器：

```
@asynccontextmanager
async def owned_token_stream(state):
    stream = token_stream(state)
    async with aclosing(stream):
        yield stream
```

`aclosing()` 在离开 `async with` 时等待 `stream.aclose()` 。因此调用者拿到第一个 token 并返回时，

README README README
'2502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

`aclosing()` 在离开 `async with` 时等待 `stream.aclose()`。因此调用者拿到第一个 token 并返回时，生成器的 `finally` 已经执行，而不是等事件循环退出时碰运气回收。

一条完整测试应检查返回值、closed 状态、关闭次数恰好为 1，以及必要的事件顺序。这个 Fake 只证明异步生成器所有权；它没有验证 HTTP 分块、SSE/JSONL framing、连接池、背压或远端断开。

ContextVar：用屏障制造确定性交错

两个请求是否隔离，不能只 `gather()` 后期待它们"碰巧交错"。可以让两个 Task 都设置请求 ID，再在 `Barrier(2)` 汇合：

```
token = request_id_var.set(request_id)
try:
    before = request_id_var.get()
    await barrier.wait()
    after = request_id_var.get()
    return before, after
finally:
    request_id_var.reset(token)
```

两个请求的结果必须分别是 `("req-A", "req-A")` 和 `("req-B", "req-B")`。调用结束后，父上下文仍应读取默认值 `"-"`，证明 token 已恢复。

这里证明的是同一事件循环中 Task 的 Context 隔离，以及函数入口正确 reset。它没有覆盖线程池、子进程、远程服务或分布式追踪传播。

11. I/O 测试要说清验证层次

"没有访问公网"和"只做了纯单元测试"不是一回事。本章在本地选择不同真实程度的边界：

测试手段	实际验证	没有验证
Pydantic 直接调用	坏参数、严格类型、未知字段、错误位置	HTTP、模型真实输出分布
`AsyncMock` 模型函数	调用参数、次数、确实 await	SDK 序列化、网络和服务端行为
HTTPX `MockTransport`	请求路径与 query、状态处理、JSON 到响应模型	DNS、socket、TLS、代理、连接池、真实网络 timeout

README README README
lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

`tmp_path` 下的 SQLite 文件	真实建表、SQL、参数绑定、读取和连接关闭	生产数据库驱动、权限、迁移、网络和锁竞争
Event 驱动 Fake	Task 状态机、取消和清理的因果顺序	操作系统或远端资源是否真正支持取消

HTTP fixture 仍创建真实 `httpx.AsyncClient`，只是把 transport 替换成进程内 handler。业务函数会真实构造 Request、处理状态码、解析 JSON 并做 Pydantic 校验，但不会创建 socket。

SQLite fixture 则在 `tmp_path` 中创建真实文件，比 `Mock(sqlite3.Connection)` 更容易发现 SQL 语法和参数绑定错误，又不会接触长期数据库。注意 sqlite3 connection 的上下文管理器负责事务提交或回滚，并不负责关闭连接；本章被测代码额外使用 `contextlib.closing()`。

12. 用一条故意失败的测试验证“测试真的会红”

只看全绿测试，可能不知道断言能否识别目标缺陷。本章保留了一个错误实现：它读取异步生成器的第一项后直接返回，没有先调用 `aclose()`。

受控失败文件是
`tests/intentional_failure_demo.py`。文件名不匹配
`test_*.py`，所以正常套件不会收集它。只有显式指定时才运行：

```
../../../.venv/bin/python -B -m pytest tests/intentional_failure_demo.py -q
```

预期命令以状态码 1 退出，并显示：

```
AssertionError: 缺陷：提前返回时没有关闭异步生成器
```

故意失败的断言仍放在 `try/finally` 中，测试最后会等待泄漏流的 `aclose()`。测试报告会变红，但不会把未关闭的异步生成器继续遗留给事件循环。

不要把它改名为 `test_intentional_failure_demo.py` 放入正常 CI。正确修复也不是删除断言，而是让资源所有者使用 `aclosing()` 或等价的明确生命周期。

13. Java 工程师最容易踩的坑

1. **在 async 测试里调用 `asyncio.run()`**：pytest-asyncio 已经拥有运行中的循环；直接 `await`。
2. **只调用 `task.cancel()`**：这只是取消请求；还要 `await task` 并断言清理和终态。
3. **用很短的 `sleep` 等任务启动**：这是调度猜测；改用 Event、Barrier、队列或调用记录。

README README README
lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

13. Java 工程师最容易踩的坑

1. **在 async 测试里调用 `asyncio.run()`**：pytest-asyncio 已经拥有运行中的循环；直接 `await`。
2. **只调用 `task.cancel()`**：这只是取消请求；还要 `await task` 并断言清理和终态。
3. **用很短的 `sleep` 等任务启动**：这是调度猜测；改用 Event、Barrier、队列或调用记录。
4. **把测试 timeout 当业务清理**：它只能让测试有限失败，不能替被测代码释放资源。
5. **让普通 Mock 返回字符串给 async 接口**：字符串不可等待；使用 AsyncMock 或 async Fake。
6. **patch 原始定义位置**：被测模块可能已经绑定旧名字；patch 实际使用位置。
7. **让 session fixture 返回可变对象**：测试会互相污染；缩小 scope 或提供副本/回滚。
8. **strict 模式下用普通装饰器声明 async fixture**：插件不会接管；使用 `@pytest_asyncio.fixture`。
9. **只断言出现 `ExceptionGroup`**：未知程序错误也能满足；继续检查异常叶子。
10. **把 MockTransport 当真实网络**：它不经过 socket；网络行为需要单独的回环或集成测试。
11. **测试结束时仍有未等待 Task**：创建者保存引用，并在成功、失败和取消路径都收集它。
12. **期待事件循环最终替你关闭流**：请求边界早已结束；资源所有者应在局部作用域明确关闭。

写异步测试前，可以依次问自己：

1. 我要验证的是结果、交互、状态转换，还是某一层真实 I/O?
2. 最小够用的替身是函数、Fake、Mock、AsyncMock，还是 MockTransport?
3. 谁创建 Task、客户端、流和数据库文件？谁负责在所有退出路径等待并关闭?
4. 执行顺序能否由 Event、Barrier 或状态证明，而不是由耗时猜测?
5. timeout 是业务合同，还是仅用于防止测试挂住?
6. 失败后是否检查了异常叶子、兄弟取消、`finally` 和零遗留任务?
7. 当前替身没有覆盖的行为，是否已经清楚写出?

14. 运行本章项目

环境和依赖

仓库根 `pyproject.toml` 要求 Python `>=3.14`。本章运行依赖为
HTTPX 0.28.1 和 Pydantic 2.13.5；开发依赖再增加 pytest 9.1.1 与 pytest-asyncio 1.4.0。

从仓库根目录安装：

```
.venv/bin/python -V
.venv/bin/python -m pip install \
```

README README README
lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

从仓库根目录安装：

```
.venv/bin/python -V
.venv/bin/python -m pip install \
  -r java_to_python/chapters/27_pytest_async/requirements-dev.txt
.venv/bin/python -m pip check
```

运行可观察演示

```
.venv/bin/python -B java_to_python/chapters/27_pytest_async/example.py
```

观察重点不是记住七段输出，而是确认：取消后 `finally` 已执行、必需任务失败会回收兄弟、并发峰值为 2、提前返回前流已关闭、两个 request id 没有串值、进程内 HTTP 和临时数据库都不会访问生产资源，以及错误实现确实暴露了未关闭状态。

运行正常测试套件

```
(cd java_to_python/chapters/27_pytest_async && \
  ../../../.venv/bin/python -B -m pytest)
```

之所以进入本章目录，是为了让 `chapter27/` 作为正常本地包出现在导入路径中。测试没有修改 `sys.path`，也没有导入名字以数字开头的章节目录。现有 `java_to_python/run_all.py` 只负责前 10 章，本章不通过它运行。

运行静态检查

```
.venv/bin/ruff check java_to_python/chapters/27_pytest_async
.venv/bin/ruff format —check java_to_python/chapters/27_pytest_async
source .venv/bin/activate
pyright java_to_python/chapters/27_pytest_async/example.py \
  java_to_python/chapters/27_pytest_async/chapter27 \
  java_to_python/chapters/27_pytest_async/tests
```

完整运行验证运行时行为，Ruff 检查格式和常规代码规则，Pyright 做静态类型检查。三者不能互相替代。

README README README
z72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

替代。

文件职责

文件	作用
`README.md`	独立教程、测试设计原则、运行方法和练习
`example.py`	打印七类可观察行为；只在最外层创建一次事件循环
`chapter27/agent_runtime.py`	本章使用的最小 Agent 风格被测模块
`tests/conftest.py`	function-scope、同步和异步 yield fixture
`tests/test_validation_and_doubles.py`	校验、参数化、fixture 和测试替身
`tests/test_async_runtime.py`	取消、TaskGroup、deadline、并发、流、ContextVar 和异常组
`tests/test_io_boundaries.py`	进程内 HTTP transport 与临时 SQLite 文件
`tests/intentional_failure_demo.py`	正常发现范围外的受控失败演示
`pytest.ini`	发现规则、strict 模式和 function loop scope
`requirements.txt`	示例与业务模块的运行依赖
`requirements-dev.txt`	运行依赖加测试依赖

本章所有正常测试都离线运行。它们没有验证公网、真实 LLM/MCP 服务、DNS、TLS、代理、付费 API、生产数据库驱动或高并发事务，也没有承诺 deadline 精确到某个墙钟毫秒。

15. 递进练习

练习 1：补齐响应模型的错误矩阵

为 `ModelReply` 增加三组参数化坏输入：缺少 `answer`、`answer` 为空、`answer` 为整数。直接调用 `model_validate()`，给每组输入可读的 id，并检查错误字段。暂时不要引入 HTTP。

练习 2：让异步清理也需要放行

给 `ToolLifecycle` 增加 `cleanup_started` 和 `cleanup_may_finish` 两个 Event。测试先等任务 started，再取消；等 cleanup_started 后先证明 Task 尚未 done，最后放行清理并等待 `CancelledError`。整个测试不使用随机 sleep，并检查资源只关闭一次。

README README README
7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-cdb898fa553df85cb8bc8f12ac08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

为 `ModelReply` 增加三组参数化坏输入：缺少 `answer`、`answer` 为空、`answer` 为整数。直接调用 `model_validate()`，给每组输入可读的 id，并检查错误字段。暂时不要引入 HTTP。

练习 2：让异步清理也需要放行

给 `ToolLifecycle` 增加 `cleanup_started` 和 `cleanup_may_finish` 两个 Event。测试先等任务 started，再取消；等 cleanup_started 后先证明 Task 尚未 done，最后放行清理并等待 `CancelledError`。
整个测试不使用随机 sleep，并检查资源只关闭一次。

练习 3：未知错误不能成为部分成功

在三个 optional tool 中让一个抛 `ToolProgramError`，另外两个停在 Event。不要把 `run_optional_tools()` 改成捕获所有异常。检查最终异常叶子、另外两个 Task 的取消和清理状态，并确保没有未等待 Task。

练习 4：增加两种 HTTP 响应失败

用两个独立 `MockTransport` handler 分别返回非法 JSON，以及合法 JSON 但字段不符合 `ModelReply`。测试应区分 JSON 解码失败和 Pydantic 校验失败，同时说明它仍没有验证真实网络。

练习 5（选学）：审计共享事件循环的所有权

只在一个新测试模块中，让两个相邻测试与一个 async fixture 使用 module loop scope。确认 fixture 在同一循环内 setup 和 teardown，并解释这个资源为什么值得共享，以及怎样避免可变状态污染。

16. 本章小结

pytest 提供发现、断言、参数化和 fixture；pytest-asyncio 负责让 coroutine 测试在明确的事件循环中运行。真正决定异步测试质量的，是下面四个问题：

1. 是否用可观察状态建立了确定的执行顺序；
2. 是否等待每个 Task 到达终态，并检查 `finally` 已完成；
3. 是否按业务语义区分部分失败、整体失败、timeout、取消和程序缺陷；
4. 是否准确说明测试替身覆盖的边界，而没有把进程内测试夸大成真实系统验证。

如果一条测试只能证明“这次运行没报错”，它还不是可靠的异步测试。如果它能明确回答“哪个状态已经发生、谁拥有资源、故障怎样传播、最后留下了什么”，它才真正构成一条回归合同。

```python
"""第 27 章的可正常导入本地包。"""

from .agent_runtime import (
    ModelReply,
    SearchArgs,
    StreamState,
    ToolBusinessError,
    ToolOutcome,
    ToolProgramError,
)

__all__: list[str] = [
    "ModelReply",
    "SearchArgs",
    "StreamState",
    "ToolBusinessError",
    "ToolOutcome",
    "ToolProgramError",
]

```

```python
"""供第 27 章测试的最小 Agent 风格业务模块。

这里把第 14、21~26 章的关键行为缩成一个本地包，避免从编号目录跨章导入。
实现只访问进程内 transport 或调用方给出的临时 SQLite 文件。
"""

import asyncio
import sqlite3
from collections.abc import (
    AsyncGenerator,
    AsyncIterator,
    Awaitable,
    Callable,
    Coroutine,
    Iterable,
    Mapping,
)
from contextlib import aclosing, asynccontextmanager, closing
from contextvars import ContextVar
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Protocol

import httpx
from pydantic import BaseModel, ConfigDict, Field, field_validator


class SearchArgs(BaseModel):
    """不可信模型参数进入工具前必须通过的运行时边界。"""

    model_config = ConfigDict(extra="forbid")

    query: str = Field(strict=True, min_length=1)
    limit: int = Field(default=3, strict=True, ge=1, le=5)

    @field_validator("query")
    @classmethod
```

```
28  class SearchArgs(BaseModel):
36      @field_validator("query")
37      @classmethod
38      def query_must_not_be_blank(cls, value: str) -> str:
39          normalized: str = value.strip()
40          if not normalized:
41              raise ValueError("query 不能只包含空白")
42          return normalized
43
44
45  class ModelReply(BaseModel):
46      """可控 HTTP 响应通过 JSON 解析后仍要验证结构。"""
47
48      model_config = ConfigDict(extra="forbid")
49
50      answer: str = Field(strict=True, min_length=1)
51
52
53  class ToolBusinessError(Exception):
54      """合同内允许成为"部分失败结果"的已知业务错误。"""
55
56
57  class ToolProgramError(RuntimeError):
58      """协议或程序不变量被破坏，必须让整个相关工作失败。"""
59
60
61  @dataclass(frozen=True, slots=True)
62  class ToolOutcome:
63      name: str
64      value: str | None
65      error: str | None
66
67      @property
68      def ok(self) -> bool:
69          return self.error is None
70
```

```python

type ToolCall = Callable[[], Coroutine[Any, Any, str]]


async def _as_optional_outcome(name: str, call: ToolCall) -> ToolOutcome:
    """只把合同明确允许的业务失败转换为数据。"""
    try:
        return ToolOutcome(name=name, value=await call(), error=None)
    except ToolBusinessError as error:
        return ToolOutcome(name=name, value=None, error=str(error))


async def run_optional_tools(tools: Mapping[str, ToolCall]) -> list[ToolOutcome]:
    """工具相互独立：保留成功和已知业务失败，未知缺陷仍传播。"""
    async with asyncio.TaskGroup() as group:
        tasks: dict[str, Task[ToolOutcome]] = {
            name: group.create_task(_as_optional_outcome(name, call), name=f"optional:{name}")
            for name, call in tools.items()
        }
    return [tasks[name].result() for name in tools]


async def run_required_tools(tools: Mapping[str, ToolCall]) -> dict[str, str]:
    """工具全部必需：任一失败时由 TaskGroup 取消并等待其兄弟。"""
    async with asyncio.TaskGroup() as group:
        tasks: dict[str, Task[str]] = {
            name: group.create_task(call(), name=f"required:{name}") for name, call in tools.items()
        }
    return {name: tasks[name].result() for name in tools}


async def run_required_tools_before(
    tools: Mapping[str, ToolCall],
    *,
    deadline: float,
) -> dict[str, str]:
    """此用同一个绝对截止时间要求任务执行与退出遵守。"""
```

```python

async def run_required_tools_before(
    tools: Mapping[str, ToolCall],
    *,
    deadline: float,
) -> dict[str, str]:
    """用同一个绝对截止时间覆盖任务组执行与退出清理。"""
    async with asyncio.timeout_at(deadline):
        return await run_required_tools(tools)


async def run_with_limit[T, R](
    items: Iterable[T],
    *,
    max_in_flight: int,
    operation: Callable[[T], Awaitable[R]],
) -> list[R]:
    """TaskGroup 管生命周期，Semaphore 只管 operation 的在途数量。"""
    if max_in_flight < 1:
        raise ValueError("max_in_flight 必须至少为 1")

    semaphore = asyncio.Semaphore(max_in_flight)

    async def call_one(item: T) -> R:
        async with semaphore:
            return await operation(item)

    async with asyncio.TaskGroup() as group:
        tasks: list[Task[R]] = [group.create_task(call_one(item)) for item in items]
    return [task.result() for task in tasks]


@dataclass(slots=True)
class ToolLifecycle:
    """用 Event 暴露任务的开始、放行和清理阶段。"""

```

```python
@dataclass(slots=True)
class ToolLifecycle:
    """用 Event 暴露任务的开始、放行和清理阶段。"""

    started: asyncio.Event = field(default_factory=asyncio.Event)
    may_finish: asyncio.Event = field(default_factory=asyncio.Event)
    cleaned: asyncio.Event = field(default_factory=asyncio.Event)
    events: list[str] = field(default_factory=list)


async def wait_for_tool(lifecycle: ToolLifecycle) -> str:
    """一个必须在正常、失败或取消路径执行 finally 的资源型操作。"""
    lifecycle.events.append("started")
    lifecycle.started.set()
    try:
        await lifecycle.may_finish.wait()
        lifecycle.events.append("completed")
        return "tool-result"
    finally:
        lifecycle.events.append("cleaned")
        lifecycle.cleaned.set()


@dataclass(slots=True)
class StreamState:
    closed: asyncio.Event = field(default_factory=asyncio.Event)
    close_count: int = 0
    events: list[str] = field(default_factory=list)


async def token_stream(state: StreamState) -> AsyncGenerator[str]:
    """本地异步生成器替身；不模拟真实网络分块、连接池或协议错误。"""
    state.events.append("stream-open")
    try:
        for token in ("agent", " ", "ready"):
            await asyncio.sleep(0)
```

```python
async def token_stream(state: StreamState) -> AsyncGenerator[str]:
    """本地异步生成器替身；不模拟真实网络分块、连接池或协议错误。"""
    state.events.append("stream-open")
    try:
        for token in ("agent", " ", "ready"):
            await asyncio.sleep(0)
            yield token
    finally:
        state.close_count += 1
        state.events.append("stream-closed")
        state.closed.set()


@asynccontextmanager
async def owned_token_stream(state: StreamState) -> AsyncIterator[AsyncGenerator[str]]:
    """创建者拥有流；消费者提前退出也会在作用域出口 await aclose。"""
    stream: AsyncGenerator[str, None] = token_stream(state)
    async with aclosing(stream):
        yield stream


async def read_first_token(state: StreamState) -> str:
    async with owned_token_stream(state) as stream:
        async for token in stream:
            return token
    raise RuntimeError("流没有产生任何 token")


async def buggy_read_first_token(state: StreamState) -> tuple[str, AsyncGenerator[str]]:
    """教学缺陷：提前返回但把仍打开的流留给调用方。

    返回流引用只是为了让受控失败测试能在 finally 中回收它，不能当作正确 API。
    """
    stream: AsyncGenerator[str, None] = token_stream(state)
    first: str = await anext(stream)
    return first, stream
```

```python
request_id_var: ContextVar[str] = ContextVar("request_id", default="-")


async def observe_request_context(
    request_id: str,
    barrier: asyncio.Barrier,
) -> tuple[str, str]:
    """两个请求在同一屏障交错，仍各自读取自己的 ContextVar 值。"""
    token: Token[str] = request_id_var.set(request_id)
    try:
        before: str = request_id_var.get()
        await barrier.wait()
        after: str = request_id_var.get()
        return before, after
    finally:
        request_id_var.reset(token)


def flatten_exception_group(group: BaseExceptionGroup[BaseException]) -> list[BaseException]:
    """递归取得异常组叶子，避免只检查最外层容器。"""
    leaves: list[BaseException] = []
    for error in group.exceptions:
        if isinstance(error, BaseExceptionGroup):
            leaves.extend(flatten_exception_group(error))
        else:
            leaves.append(error)
    return leaves


def classify_failure(error: BaseException) -> str:
    """分类顺序让取消不被宽泛的普通错误分支吞掉。"""
    if isinstance(error, asyncio.CancelledError):
        return "cancelled"
    if isinstance(error, ToolBusinessError):
        return "business"
    if isinstance(error, TimeoutError):
```

```python
def classify_failure(error: BaseException) -> str:
    """分类顺序让取消不被宽泛的普通错误分支吞掉。"""
    if isinstance(error, asyncio.CancelledError):
        return "cancelled"
    if isinstance(error, ToolBusinessError):
        return "business"
    if isinstance(error, TimeoutError):
        return "timeout"
    return "program-defect"


class AsyncTextCall(Protocol):
    async def __call__(self, prompt: str) -> str: ...


async def ask_model(model_call: AsyncTextCall, prompt: str) -> str:
    """依赖必须返回 awaitable；普通 Mock 的字符串返回值不能被 await。"""
    return await model_call(prompt)


async def call_model_api(prompt: str) -> str:
    """真实项目中的外部边界；离线测试应在使用位置替换它。"""
    raise RuntimeError(f"离线示例不会调用真实模型：{prompt}")


async def answer_with_default_model(prompt: str) -> str:
    return await call_model_api(prompt)

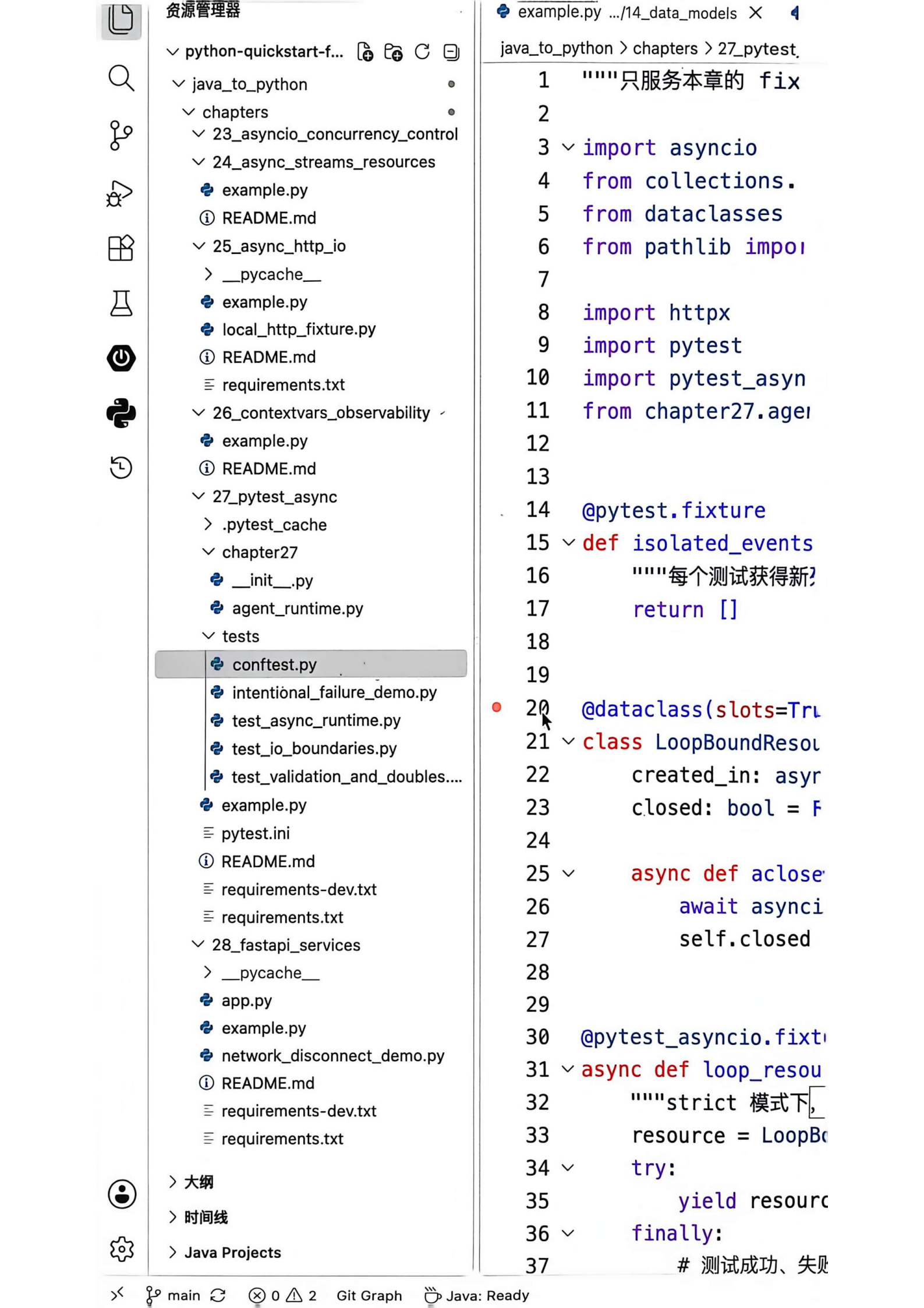

async def fetch_model_reply(
    client: httpx.AsyncClient,
    raw_args: object,
) -> ModelReply:
    """验证参数，再通过调用方拥有的 AsyncClient 访问 HTTP 边界。"""
    args: SearchArgs = SearchArgs.model_validate(raw_args)
    response: Response = await client.get("/model", params={"query": args.query, "limit": args.limit})
    response.raise_for_status()
```

```python
async def fetch_model_reply(
    client: httpx.AsyncClient,
    raw_args: object,
) -> ModelReply:
    """验证参数，再通过调用方拥有的 AsyncClient 访问 HTTP 边界。"""
    args: SearchArgs = SearchArgs.model_validate(raw_args)
    response: Response = await client.get("/model", params={"query": args.query, "limit": args.limit})
    response.raise_for_status()
    return ModelReply.model_validate(response.json())


def initialize_notes(database_path: Path) -> None:
    """创建调用方指定的 SQLite 文件；测试会传入 tmp_path 下的路径。"""
    with closing(sqlite3.connect(database_path)) as connection, connection:
        connection.execute("CREATE TABLE notes (id INTEGER PRIMARY KEY, body TEXT NOT NULL)")
        connection.executemany(
            "INSERT INTO notes (id, body) VALUES (?, ?)",
            ((1, "pytest fixture"), (2, "async cleanup")),
        )


def read_note(database_path: Path, note_id: int) -> str | None:
    with closing(sqlite3.connect(database_path)) as connection:
        row: Any = connection.execute(
            "SELECT body FROM notes WHERE id = ?",
            (note_id,),
        ).fetchone()
    return None if row is None else str(row[0])

```


资源管理器
python-quickstart-f...
java_to_python
chapters
23_asyncio_concurrency_control
24_async_streams_resources
example.py
README.md
25_async_http_io
__pycache__
example.py
local_http_fixture.py
README.md
requirements.txt
26_contextvars_observability
example.py
README.md
27_pytest_async
.pytest_cache
chapter27
__init__.py
agent_runtime.py
tests
conftest.py
intentional_failure_demo.py
test_async_runtime.py
test_io_boundaries.py
test_validation_and_doubles....
example.py
pytest.ini
README.md
requirements-dev.txt
requirements.txt
28_fastapi_services
__pycache__
app.py
example.py
network_disconnect_demo.py
README.md
requirements-dev.txt
requirements.txt
大纲
时间线
Java Projects
example.py .../14_data_models
java_to_python > chapters > 27_pytest_
1 """只服务本章的 fix
2
3 import asyncio
4 from collections.
5 from dataclasses
6 from pathlib impo
7
8 import httpx
9 import pytest
10 import pytest_asyn
11 from chapter27.age
12
13
14 @pytest.fixture
15 def isolated_events
16 """每个测试获得新
17 return []
18
19
20 @dataclass(slots=Tr
21 class LoopBoundResou
22 created_in: asyr
23 closed: bool = F
24
25 async def aclose
26 await asynci
27 self.closed
28
29
30 @pytest_asyncio.fixt
31 async def loop_resou
32 """strict 模式下,
33 resource = LoopBo
34 try:
35 yield resourc
36 finally:
37 # 测试成功、失败
main
0 2
Git Graph
Java: Ready

example.py .../14_data_models × example.py .../15_protocols_generics × example.py .../16_decorators × *conftest.py* ×

java_to_python > chapters > 27_pytest_async > tests > conftest.py > ...

```python
"""只服务本章的 fixture；默认 function scope 保持测试隔离。"""

import asyncio
from collections.abc import AsyncIterator, Iterator
from dataclasses import dataclass
from pathlib import Path

import httpx
import pytest
import pytest_asyncio
from chapter27.agent_runtime import initialize_notes


@pytest.fixture
def isolated_events() -> list[str]:
    """每个测试获得新列表，避免共享可变状态造成顺序依赖。"""
    return []


@dataclass(slots=True)
class LoopBoundResource:
    created_in: asyncio.AbstractEventLoop
    closed: bool = False

    async def aclose(self) -> None:
        await asyncio.sleep(0)
        self.closed = True


@pytest_asyncio.fixture
async def loop_resource() -> AsyncIterator[LoopBoundResource]:
    """strict 模式下，async fixture 明确使用 pytest_asyncio.fixture。"""
    resource = LoopBoundResource(asyncio.get_running_loop())
    try:
        yield resource
    finally:
        # 测试成功、失败或被跳过，fixture 都会从 yield 后继续做清理。
```

Ready

```python
@pytest_asyncio.fixture
async def loop_resource() -> AsyncIterator[LoopBoundResource]:
    """strict 模式下, async fixture 明确使用 pytest_asyncio.fixture。"""
    resource = LoopBoundResource(asyncio.get_running_loop())
    try:
        yield resource
    finally:
        # 测试成功、失败或被跳过，fixture 都会从 yield 后继续做清理。
        await resource.aclose()
        assert resource.closed


@pytest_asyncio.fixture
async def model_client() -> AsyncIterator[httpx.AsyncClient]:
    """进程内 HTTP transport: 验证请求/响应代码，不创建 socket。"""

    def handle_request(request: httpx.Request) -> httpx.Response:
        assert request.url.params["query"] == "asyncio"
        assert request.url.params["limit"] == "2"
        return httpx.Response(200, json={"answer": "使用结构化并发"})

    transport = httpx.MockTransport(handle_request)
    async with httpx.AsyncClient(
        transport=transport,
        base_url="https://model.test",
    ) as client:
        yield client
    assert client.is_closed


@pytest.fixture
def note_database(tmp_path: Path) -> Iterator[Path]:
    """同步 yield fixture: 测试借用文件, yield 后由拥有者删除。"""
    database_path: Path = tmp_path / "agent-notes.sqlite3"
    initialize_notes(database_path)
    try:
```

```
43  async def model_client() -> AsyncIterator[httpx.AsyncClient]:
50
51      transport = httpx.MockTransport(handle_request)
52      async with httpx.AsyncClient(
53          transport=transport,
54          base_url="https://model.test",
55      ) as client:
56          yield client
57      assert client.is_closed
58
59
60  @pytest.fixture
61  def note_database(tmp_path: Path) -> Iterator[Path]:
62      """同步 yield fixture：测试借用文件，yield 后由拥有者删除。"""
63      database_path: Path = tmp_path / "agent-notes.sqlite3"
64      initialize_notes(database_path)
65      try:
66          yield database_path
67      finally:
68          database_path.unlink(missing_ok=True)
69          assert not database_path.exists()
70
```

```python
"""受控失败演示；文件名不匹配 test_*.py，正常测试套件不会收集它。"""

import asyncio

import pytest
from chapter27.agent_runtime import StreamState, buggy_read_first_token


@pytest.mark.asyncio
async def test_buggy_reader_should_close_stream_before_returning() -> None:
    state = StreamState()
    first: str, leaked_stream: AsyncGenerator[str, None] = await buggy_read_first_token(state)

    try:
        assert first == "agent"
        # 这里会失败，准确指出真实缺陷：函数已返回，但流仍停在 yield。
        assert state.closed.is_set(), "缺陷：提前返回时没有关闭异步生成器"
    finally:
        # 即使断言故意失败，也显式回收流，不把 pending async generator 留给循环退出。
        await asyncio.wait_for(leaked_stream.aclose(), timeout=1.0)

```

```python
"""异步故障、取消、截止时间、并发、流和 ContextVar 的稳定测试。"""

import asyncio
from dataclasses import dataclass, field

import pytest
from chapter27.agent_runtime import (
    StreamState,
    ToolBusinessError,
    ToolLifecycle,
    ToolProgramError,
    classify_failure,
    flatten_exception_group,
    observe_request_context,
    read_first_token,
    request_id_var,
    run_optional_tools,
    run_required_tools,
    run_required_tools_before,
    run_with_limit,
    wait_for_tool,
)
from conftest import LoopBoundResource


async def wait_until_set(*events: asyncio.Event) -> None:
    """一秒只是防悬挂护栏；测试顺序由 Event 建立，不靠这段时间碰运气。"""
    async with asyncio.timeout(1.0):
        for event in events:
            await event.wait()


@dataclass(slots=True)
class ControlledTool:
    value: str
    error: Exception | None = None
    started: asyncio.Event = field(default_factory=asyncio.Event)
```

```python
@dataclass(slots=True)
class ControlledTool:
    value: str
    error: Exception | None = None
    started: asyncio.Event = field(default_factory=asyncio.Event)
    release: asyncio.Event = field(default_factory=asyncio.Event)
    cleaned: asyncio.Event = field(default_factory=asyncio.Event)
    was_cancelled: bool = False

    async def __call__(self) -> str:
        self.started.set()
        try:
            await self.release.wait()
            if self.error is not None:
                raise self.error
            return self.value
        except asyncio.CancelledError:
            self.was_cancelled = True
            raise
        finally:
            self.cleaned.set()


@pytest.mark.asyncio
async def test_pytest_owns_running_loop(loop_resource: LoopBoundResource) -> None:
    running_loop: AbstractEventLoop = asyncio.get_running_loop()

    assert loop_resource.created_in is running_loop
    assert not loop_resource.closed


@pytest.mark.asyncio
async def test_cancellation_waits_for_start_then_checks_finally() -> None:
    lifecycle = ToolLifecycle()
    task: Task[str] = asyncio.create_task(wait_for_tool(lifecycle), name="cancel-cleanup-test")

```

```python
@pytest.mark.asyncio
async def test_cancellation_waits_for_start_then_checks_finally() -> None:
    lifecycle = ToolLifecycle()
    task: Task[str] = asyncio.create_task(wait_for_tool(lifecycle), name="cancel-cleanup-test")

    await wait_until_set(lifecycle.started)
    task.cancel("测试主动取消")

    with pytest.raises(asyncio.CancelledError):
        async with asyncio.timeout(1.0):
            await task

    assert lifecycle.cleaned.is_set()
    assert lifecycle.events == ["started", "cleaned"]
    assert task.cancelled()


@pytest.mark.asyncio
async def test_optional_tools_keep_expected_partial_failure() -> None:
    search = ControlledTool("docs")
    record = ControlledTool("unused", ToolBusinessError("记录不存在"))
    tools: dict[str, ControlledTool] = {"search": search, "record": record}
    request: Task[list[ToolOutcome]] = asyncio.create_task(run_optional_tools(tools))

    await wait_until_set(search.started, record.started)
    search.release.set()
    record.release.set()
    outcomes: list[ToolOutcome] = await request

    assert [outcome.ok for outcome in outcomes] == [True, False]
    assert outcomes[0].value == "docs"
    assert outcomes[1].error == "记录不存在"
    assert search.cleaned.is_set() and record.cleaned.is_set()


@pytest.mark.asyncio
async def test_required_failure_cancels_siblings_and_preserves_group() -> None:
```

```python
@pytest.mark.asyncio
async def test_required_failure_cancels_siblings_and_preserves_group() -> None:
    waiting_a = ControlledTool("never-a")
    failing = ControlledTool("never-failure", ToolProgramError("响应协议损坏"))
    waiting_b = ControlledTool("never-b")
    tools: dict[str, ControlledTool] = {"a": waiting_a, "failure": failing, "b": waiting_b}
    request: Task[dict[str, str]] = asyncio.create_task(run_required_tools(tools))

    await wait_until_set(waiting_a.started, failing.started, waiting_b.started)
    failing.release.set()

    with pytest.raises(ExceptionGroup) as raised:
        async with asyncio.timeout(1.0):
            await request

    leaves: list[BaseException] = flatten_exception_group(raised.value)
    assert len(leaves) == 1
    assert isinstance(leaves[0], ToolProgramError)
    assert str(leaves[0]) == "响应协议损坏"
    assert waiting_a.was_cancelled and waiting_b.was_cancelled
    assert all(tool.cleaned.is_set() for tool in (waiting_a, failing, waiting_b))


@pytest.mark.asyncio
async def test_whole_deadline_cancels_and_cleans_pending_tool() -> None:
    pending = ControlledTool("never")
    loop: AbstractEventLoop = asyncio.get_running_loop()
    request: Task[dict[str, str]] = asyncio.create_task(
        run_required_tools_before(
            {"pending": pending},
            deadline=loop.time() + 0.2,
        )
    )

    await wait_until_set(pending.started)
    with pytest.raises(TimeoutError):
```

```python
@pytest.mark.asyncio
async def test_whole_deadline_cancels_and_cleans_pending_tool() -> None:
    pending = ControlledTool("never")
    loop: AbstractEventLoop = asyncio.get_running_loop()
    request: Task[dict[str, str]] = asyncio.create_task(
        run_required_tools_before(
            {"pending": pending},
            deadline=loop.time() + 0.2,
        )
    )

    await wait_until_set(pending.started)
    with pytest.raises(TimeoutError):
        async with asyncio.timeout(1.0):
            await request

    assert pending.was_cancelled
    assert pending.cleaned.is_set()


@dataclass(slots=True)
class ConcurrencyProbe:
    expected_peak: int
    active: int = 0
    max_active: int = 0
    peak_reached: asyncio.Event = field(default_factory=asyncio.Event)
    release: asyncio.Event = field(default_factory=asyncio.Event)

    async def call(self, item: int) -> int:
        self.active += 1
        self.max_active = max(self.max_active, self.active)
        if self.active == self.expected_peak:
            self.peak_reached.set()
        try:
            await self.release.wait()
            return item * 10
        finally:
```

```
143 class ConcurrencyProbe:
150     async def call(self, item: int) -> int:
155         try:
156             await self.release.wait()
157             return item * 10
158         finally:
159             self.active -= 1
160
161
162 @pytest.mark.asyncio
163 async def test_concurrency_limit_uses_observed_peak_not_elapsed_time() -> None:
164     probe = ConcurrencyProbe(expected_peak=2)
165     request: Task[list[int]] = asyncio.create_task(run_with_limit(range(6), max_in_flight=2, operation=probe.
166
167     await wait_until_set(probe.peak_reached)
168     assert probe.active == 2
169     assert probe.max_active == 2
170     probe.release.set()
171     results: list[int] = await request
172
173     assert results == [0, 10, 20, 30, 40, 50]
174     assert probe.max_active == 2
175     assert probe.active == 0
176
177
178 @pytest.mark.asyncio
179 async def test_early_stream_exit_closes_owned_generator() -> None:
180     state = StreamState()
181
182     first: str = await read_first_token(state)
183
184     assert first == "agent"
185     assert state.closed.is_set()
186     assert state.close_count == 1
187     assert state.events == ["stream-open", "stream-closed"]
188
```

```python

@pytest.mark.asyncio
async def test_contextvar_values_do_not_cross_requests() -> None:
    barrier = asyncio.Barrier(2)

    first: tuple[str, str], second: tuple[str, str] = await asyncio.gather(
        observe_request_context("req-A", barrier),
        observe_request_context("req-B", barrier),
    )

    assert first == ("req-A", "req-A")
    assert second == ("req-B", "req-B")
    assert request_id_var.get() == "-"


def test_exception_group_leaves_are_classified_by_meaning() -> None:
    grouped: BaseExceptionGroup[ToolBusinessError |... = BaseExceptionGroup(
        "request failures",
        [
            ToolBusinessError("输入不可用"),
            ExceptionGroup("nested", [TimeoutError("整体超时"), ValueError("程序缺陷")]),
            asyncio.CancelledError("用户停止"),
        ],
    )

    categories: list[str] = [classify_failure(error) for error in flatten_exception_group(grouped)]

    assert categories == ["business", "timeout", "program-defect", "cancelled"]

```

```python
"""HTTP transport 与临时 SQLite 文件分别验证不同边界。"""

from pathlib import Path

import httpx
import pytest
from chapter27.agent_runtime import fetch_model_reply, read_note


@pytest.mark.asyncio
async def test_http_mock_transport_checks_request_and_response_model(
    model_client: httpx.AsyncClient,
) -> None:
    reply: ModelReply = await fetch_model_reply(
        model_client,
        {"query": "asyncio", "limit": 2},
    )

    assert reply.answer == "使用结构化并发"


def test_sqlite_uses_a_per_test_temporary_file(note_database: Path, tmp_path: Path) -> None:
    assert note_database.parent == tmp_path
    assert note_database.is_file()
    assert read_note(note_database, 1) == "pytest fixture"
    assert read_note(note_database, 999) is None

```

```python
"""从同步数据校验开始，再比较 fixture、Mock 与 monkeypatch。"""

from typing import cast
from unittest.mock import AsyncMock, Mock

import pytest
from chapter27 import agent_runtime
from chapter27.agent_runtime import AsyncTextCall, SearchArgs, ask_model
from pydantic import ValidationError


def test_model_accepts_valid_search_args() -> None:
    args: SearchArgs = SearchArgs.model_validate({"query": "  asyncio  ", "limit": 2})

    assert args.query == "asyncio"
    assert args.limit == 2


@pytest.mark.parametrize(
    ("raw_args", "bad_field"),
    [
        pytest.param({"query": "", "limit": 2}, "query", id="empty-query"),
        pytest.param({"query": "   ", "limit": 2}, "query", id="blank-query"),
        pytest.param({"query": "ok", "limit": 0}, "limit", id="limit-too-small"),
        pytest.param({"query": "ok", "limit": "2"}, "limit", id="strict-integer"),
        pytest.param({"query": "ok", "limit": 2, "extra": True}, "extra", id="extra-field"),
    ],
)
def test_model_rejects_bad_search_args(raw_args: object, bad_field: str) -> None:
    with pytest.raises(ValidationError) as raised:
        SearchArgs.model_validate(raw_args)

    error_locations: set[str] = {str(error["loc"][0]) for error in raised.value.errors()}
    assert bad_field in error_locations


def test_function_fixture_starts_empty_a(isolated_events: list[str]) -> None:
```

```python
def test_function_fixture_starts_empty_a(isolated_events: list[str]) -> None:
    assert isolated_events == []
    isolated_events.append("只属于测试 A")


def test_function_fixture_starts_empty_b(isolated_events: list[str]) -> None:
    # 即使 A 先运行，这里仍是新对象；测试不依赖收集或执行顺序。
    assert isolated_events == []


@pytest.mark.asyncio
async def test_asyncmock_is_awaited() -> None:
    model_call = AsyncMock(return_value="离线模型结果")

    result: str = await ask_model(model_call, "解释 TaskGroup")

    assert result == "离线模型结果"
    model_call.assert_awaited_once_with("解释 TaskGroup")


@pytest.mark.asyncio
async def test_plain_mock_return_value_is_not_awaitable() -> None:
    plain_mock = Mock(return_value="普通字符串")
    # cast 只能说服类型检查器，不能把 str 在运行时变成 awaitable。
    claimed_async_call: AsyncTextCall = cast(AsyncTextCall, plain_mock)

    with pytest.raises(TypeError, match="await"):
        await ask_model(claimed_async_call, "这个调用必须可等待")

    plain_mock.assert_called_once_with("这个调用必须可等待")


@pytest.mark.asyncio
async def test_monkeypatch_replaces_dependency_at_use_site(
    monkeypatch: pytest.MonkeyPatch,
) -> None:
```

```
58  async def test_plain_mock_return_value_is_not_awaitable() -> None:
64          await ask_model(claimed_async_call, "这个调用必须可等待")
65
66      plain_mock.assert_called_once_with("这个调用必须可等待")
67
68
69  @pytest.mark.asyncio
70  async def test_monkeypatch_replaces_dependency_at_use_site(
71      monkeypatch: pytest.MonkeyPatch,
72  ) -> None:
73      fake_api = AsyncMock(return_value="patched-result")
74      monkeypatch.setattr(agent_runtime, "call_model_api", fake_api)
75
76      result: str = await agent_runtime.answer_with_default_model("离线请求")
77
78      assert result == "patched-result"
79      fake_api.assert_awaited_once_with("离线请求")
80
```

```python
"""第 27 章：离线观察 pytest 将要验证的异步行为。

从仓库根目录运行：
    .venv/bin/python -B java_to_python/chapters/27_pytest_async/example.py

这个入口拥有自己的事件循环，所以只在最外层调用一次 asyncio.run()；pytest 异步
测试的事件循环则由 pytest-asyncio 拥有，测试函数内部不会再次调用 asyncio.run()。
"""

# 阅读顺序：先看同步参数校验，再依次观察取消、TaskGroup、并发上限、流与
# ContextVar、I/O 边界和受控缺陷。这个文件负责把现象打印出来；真正可重复的
# 断言位于 tests/。外部边界都被限制在进程内或临时资源中，因此运行示例不会访问公网。
#
# asyncio 提供事件循环、Task 和 Event/Barrier 等同步原语；tempfile 与 Path
# 用于创建会自动删除的临时数据库目录。dataclass/field 则用少量代码声明后面的
# 有状态 Fake，作用类似 Java 测试里的小型 POJO，但 Python 不会据此做静态类型强制。
import asyncio
import tempfile
from dataclasses import dataclass, field
from pathlib import Path

# httpx 在第 6 段构造进程内 HTTP 客户端。chapter27.agent_runtime 是本章真正的
# 被测业务模块：入口只准备输入、控制执行阶段并展示它的公开结果。
import httpx
from chapter27.agent_runtime import (
    SearchArgs,
    StreamState,
    ToolBusinessError,
    ToolLifecycle,
    ToolProgramError,
    buggy_read_first_token,
    fetch_model_reply,
    flatten_exception_group,
    initialize_notes,
    observe_request_context,
    read_first_token,
    read_note,
```

```
    read_first_token,
    read_note,
    request_id_var,
    run_optional_tools,
    run_required_tools,
    run_with_limit,
    wait_for_tool,
)

# Pydantic 在数据不满足 SearchArgs 约束时抛出这个异常；它包含结构化错误列表，
# 不必依赖整段、可能随版本调整的错误文本来判断哪个字段失败。
from pydantic import ValidationError


# title: str 与 -> None 是类型提示，分别表示调用方应传字符串、函数不返回业务值；
# 它们主要服务阅读和静态检查，不像 Java 方法签名那样由解释器强制执行。
def print_section(title: str) -> None:
    # f-string 会计算花括号中的表达式再嵌入文本。这里先把 "=" 重复 72 次，
    # 再在同一次 print 中用 \n 排出空行、分隔线、标题和第二条分隔线。
    print(f"\n{'=' * 72}\n{title}\n{'=' * 72}")


# 先用同步例子建立最小测试合同：同一个模型对合法字典返回规范化对象，
# 对不可信的错误字典则抛 ValidationError。这里尚未创建事件循环。
def demonstrate_sync_validation() -> None:
    print_section("1. 同步起点：模型接受好参数、拒绝坏参数")

    # model_validate 在运行时读取这个普通 dict。SearchArgs 的校验器会去掉 query
    # 两端空白，所以 args.query 是 "asyncio"；args 是模型实例，不再是原始字典。
    # model_dump() 又产生适合打印/序列化的新字典，可观察 query 的规范化结果。
    args: SearchArgs = SearchArgs.model_validate({"query": "  asyncio  ", "limit": 2})
    print("规范化后的参数:", args.model_dump())

    # try 只包住预期可能失败的调用，避免把准备或打印阶段的同类异常误当成成功案例。
    # 这份输入同时包含三个问题：query 只有空白、严格整数 limit 不接受字符串 "2"、
    # 模型配置禁止 unknown 额外字段，因此一次校验可以收集多个字段错误。
```

```python
def demonstrate_sync_validation() -> None:

    # try 只包住预期可能失败的调用，避免把准备或打印阶段的同类异常误当成成功案例。
    # 这份输入同时包含三个问题：query 只有空白、严格整数 limit 不接受字符串 "2"、
    # 模型配置禁止 unknown 额外字段，因此一次校验可以收集多个字段错误。
    try:
        SearchArgs.model_validate({"query": " ", "limit": "2", "unknown": True})
    except ValidationError as error:
        # error.errors() 返回“每项描述一个校验错误”的字典列表。列表推导式逐项
        # 取出 loc 元组，得到失败位置，而不是把整段错误消息当成不稳定的断言依据。
        locations: list[tuple[int | str, ...]] = [item["loc"] for item in error.errors()]
        print("坏参数的错误位置:", locations)


# async def 调用后先产生 coroutine 对象，只有被 await 或包装为 Task 才真正向前执行。
# 本段展示稳定的取消顺序：确认任务已进入等待点，再请求取消，最后 await 到清理完成。
async def demonstrate_cancellation_cleanup() -> None:
    print_section("2. 取消：先确认已启动，再取消、等待并检查 finally")

    # ToolLifecycle 内的 Event 是阶段信号，events 列表则保存可读的先后记录。
    # create_task 把 coroutine 安排到当前事件循环并立即返回 Task；保留 task 引用意味着
    # 当前函数仍负责等待它的最终结果。name 只用于调试，不影响调度。
    lifecycle = ToolLifecycle()
    task: Task[str] = asyncio.create_task(wait_for_tool(lifecycle), name="demo-cancellable-tool")

    # wait_for_tool 会先记录 "started"、设置 started，再阻塞于 may_finish。
    # 等 Event 比 sleep 若干毫秒更可靠：恢复到这里时，可以确定目标已进入可取消区域。
    await lifecycle.started.wait()
    print("取消前事件:", lifecycle.events)

    # cancel() 只是向 Task 提出取消请求，并把消息附到随后注入的 CancelledError；
    # 它不会像强行终止 Java 线程那样同步杀死代码。真正的取消在 Task 下次运行时传播。
    task.cancel("演示用户停止")
    try:
        # await 不只是“拿结果”，还把调用方与子任务的结束建立先后关系：
        # wait_for_tool 的 finally 已运行完，控制流才会进入下面的 except。
        await task
```

```python
async def demonstrate_cancellation_cleanup() -> None:
    try:
        # await 不只是“拿结果”，还把调用方与子任务的结束建立先后关系：
        # wait_for_tool 的 finally 已运行完，控制流才会进入下面的 except。
        await task
    except asyncio.CancelledError:
        # CancelledError 代表协作式取消的正常传播，这里只为展示而确认它；
        # 业务代码通常完成本层清理后还应继续向上抛，不能用宽泛 except 吞掉取消。
        print("调用方收到了 CancelledError")

    # is_set() 读取 Event 当前状态；cancelled() 只有在 Task 最终以取消结束后才为 True。
    # 最后的 assert 是脚本内的安全检查，正式 pytest 测试会把这些结论拆得更易诊断。
    print("取消后事件:", lifecycle.events)
    print("清理完成 / Task 已取消:", lifecycle.cleaned.is_set(), task.cancelled())
    assert lifecycle.events == ["started", "cleaned"]


# 这是用来驱动并发路径的可控 Fake，而不是生产工具。
# @dataclass 自动生成初始化方法等样板代码；slots=True 不再为每个实例创建普通 __dict__，
# 字段集合因而固定。它可类比精简的 Java 测试 POJO，但类型提示仍不是运行时类型检查。
@dataclass(slots=True)
class DemoTool:
    # value 是成功时返回的文本；error 为 None 表示成功，否则放行后抛出指定异常。
    # Exception | None 是联合类型提示，可读作“Exception 或空值 None”。
    value: str
    error: Exception | None = None

    # default_factory 会为每个 DemoTool 实例分别调用 asyncio.Event，避免多个实例错误地
    # 共享同一个可变 Event。三个信号依次表示：已启动、允许继续、已完成 finally 清理。
    started: asyncio.Event = field(default_factory=asyncio.Event)
    release: asyncio.Event = field(default_factory=asyncio.Event)
    cleaned: asyncio.Event = field(default_factory=asyncio.Event)
    cancelled: bool = False

    # 定义 __call__ 后，实例本身就能像函数一样写成 await tool()；run_*_tools 正是以
    # 这种统一调用接口接收不同 Fake。-> str 描述 await 完成后得到的值，而非 coroutine 本身。
    async def __call__(self) -> str:
```

```
120  class DemoTool:
133      # 定义 __call__ 后，实例本身就能像函数一样写成 await tool()；run_*_tools 正是以
134      # 这种统一调用接口接收不同 Fake。-> str 描述 await 完成后得到的值，而非 coroutine 本身。
135      async def __call__(self) -> str:
136          # 先发 started，再等待测试/演示方打开 release 闸门，使外部能精确控制交错顺序。
137          self.started.set()
138          try:
139              await self.release.wait()
140              # is not None 检查是否确实配置了异常对象；不能写简单的 if self.error 来猜真值。
141              if self.error is not None:
142                  raise self.error
143              return self.value
144          except asyncio.CancelledError:
145              # 记录“确实观察到取消”，随后裸 raise 原异常。若这里不重新抛出，TaskGroup
146              # 可能把已取消的工具误认为正常完成。
147              self.cancelled = True
148              raise
149          finally:
150              # 正常返回、配置异常和取消三条路径都会执行 finally，因此 cleaned 表示
151              # 生命周期已经收尾；真实实现会在这里释放连接、流或锁。
152              self.cleaned.set()
153
154
155  # 两类合同不能混在一起：可选工具允许把已知业务错误保存在结果中；
156  # 必需工具遇到程序缺陷时，整个结构化并发作用域都必须失败并回收兄弟任务。
157  async def demonstrate_partial_and_total_failure() -> None:
158      print_section("3. 部分失败与整体失败是两种合同")
159
160      # 两个独立实例分别代表成功工具和预期内的业务失败。"unused" 不会成为结果，
161      # 因为 expected_failure 在 release 后会先抛 ToolBusinessError。
162      ok = DemoTool("文档结果")
163      expected_failure = DemoTool("unused", ToolBusinessError("客户记录不存在"))
164
165      # 字典把工具名映射到可异步调用的 DemoTool；create_task 让整组工作在后台推进，
166      # 当前函数才能同时等待两个 started，并决定何时放行它们。
167      optional_task: Task[list[ToolOutcome]] = asyncio.create_task(
168          run_optional_tools({"search": ok, "record": expected_failure})
```

```python
async def demonstrate_partial_and_total_failure() -> None:

    # 字典把工具名映射到可异步调用的 DemoTool；create_task 让整组工作在后台推进，
    # 当前函数才能同时等待两个 started，并决定何时放行它们。
    optional_task: Task[list[ToolOutcome]] = asyncio.create_task(
        run_optional_tools({"search": ok, "record": expected_failure})
    )
    await ok.started.wait()
    await expected_failure.started.wait()

    # Event.set() 是同步方法：信号一旦置位，所有当前和以后等待者都可继续，无需 await。
    ok.release.set()
    expected_failure.release.set()

    # run_optional_tools 只把 ToolBusinessError 转成 error 字段；成功项保留 value，
    # 因而 await 返回的是包含两种 outcome 的列表，而不是抛出整个请求错误。
    outcomes: list[ToolOutcome] = await optional_task
    print("可选工具结果:", outcomes)

    # 第二组中 waiting 永远不会由本函数主动 release；defect 被放行并抛程序错误。
    # 这刻意制造“一个成员失败、另一个兄弟仍在等待”的 TaskGroup 场景。
    waiting = DemoTool("不会返回")
    defect = DemoTool("不会返回", ToolProgramError("响应协议损坏"))
    required_task: Task[dict[str, str]] = asyncio.create_task(run_required_tools({"waiting": waiting, "defect": defe
    await waiting.started.wait()
    await defect.started.wait()
    defect.release.set()
    try:
        # run_required_tools 内的 TaskGroup 看到 defect 失败后，会取消 waiting，并等待两者
        # 的 finally 完成；退出 TaskGroup 时再把未处理错误放入 ExceptionGroup 抛给这里。
        await required_task
    except ExceptionGroup as group:
        # ExceptionGroup 是异常树的外壳。递归展开叶子后才能确认真正原因确实是
        # ToolProgramError，而不是只满足“某个组失败了”这种过宽结论。
        leaves: list[BaseException] = flatten_exception_group(group)
        print("整体失败的异常叶子:", [type(error).__name__ for error in leaves])
```

```
async def demonstrate_partial_and_total_failure() -> None:

    # 能运行到此处时 required_task 已结束；waiting.cancelled 证明它收到取消，
    # 两个 cleaned 状态则证明 TaskGroup 没在兄弟还占用资源时提前返回。
    print("等待中的兄弟被取消 / 两者都清理:", waiting.cancelled, waiting.cleaned.is_set())
    assert waiting.cancelled and waiting.cleaned.is_set() and defect.cleaned.is_set()


# 这个探针把"并发是否超过 2"变成可直接读取的状态，避免用总耗时反推并发度。
@dataclass(slots=True)
class DemoConcurrencyProbe:
    expected_peak: int
    # active 是当前正在 operation 内的数量，max_active 保存运行期间见过的最大值。
    active: int = 0
    max_active: int = 0

    # reached_peak 通知观察者已经占满预期槽位；release 在观察完成后一次性放行任务。
    reached_peak: asyncio.Event = field(default_factory=asyncio.Event)
    release: asyncio.Event = field(default_factory=asyncio.Event)

    async def call(self, item: int) -> int:
        # asyncio 在单线程事件循环中只会于 await 等让出点切换 Task；这三步之间没有
        # await，所以更新计数并检查峰值时不会被另一个 coroutine 插入执行。
        self.active += 1
        self.max_active = max(self.max_active, self.active)
        if self.active == self.expected_peak:
            self.reached_peak.set()
        try:
            # 所有已取得 Semaphore 槽位的调用停在同一个闸门，便于中途读取真实在途数。
            await self.release.wait()
            return item * 10
        finally:
            # 即便等待期间被取消，也归还探针自己的 active 状态；Semaphore 的槽位则由
            # run_with_limit 内的 async with 负责归还。
            self.active -= 1
```

```python
async def demonstrate_concurrency_limit() -> None:
    print_section("4. 并发：断言可观察的最大在途数，不比较毫秒")
    probe = DemoConcurrencyProbe(expected_peak=2)

    # range(5) 按需提供 0～4；run_with_limit 会为各项安排工作，但 Semaphore 保证同一时刻
    # 最多两个 call 进入受保护区。包装为 Task 后，当前函数可在请求完成前检查中间状态。
    request: Task[list[int]] = asyncio.create_task(run_with_limit(range(5), max_in_flight=2, operation=probe.call))

    # reached_peak 置位说明两个槽位都已被占用。release 尚未打开，所以此刻稳定地看到
    # active/max_active 都是 2，而不是依赖某台机器“应该在几毫秒内跑到这里”。
    await probe.reached_peak.wait()
    print("闸门关闭时 active / max_active:", probe.active, probe.max_active)

    # Event 保持置位；首批任务结束后，后续取得槽位的调用也会直接通过同一闸门。
    probe.release.set()
    results: list[int] = await request

    # TaskGroup 最终按输入创建顺序读取各 Task 结果，所以列表为 0、10、20、30、40；
    # active 回到 0 则证明所有 call 的 finally 都执行完毕。
    print("结果 / 最终 active:", results, probe.active)
    assert probe.max_active == 2 and probe.active == 0


# 把两个独立概念放在一段展示：资源型异步流由创建者显式关闭；ContextVar 则让
# 同一事件循环中交错执行的请求各自看到自己的上下文值。
async def demonstrate_stream_and_context() -> None:
    print_section("5. 提前退出流与 ContextVar 请求隔离")

    # read_first_token 只消费第一个 "agent" 就 return，但其内部 aclosing 会先 await
    # 异步生成器的 aclose()。因此函数返回到这里时，finally 已设置 closed，而非稍后碰运气回收。
    stream_state = StreamState()
    first: str = await read_first_token(stream_state)
    print("只读取:", first, "; 返回前流已关闭:", stream_state.closed.is_set())

    # Barrier(2) 要求两个参与者都到达 wait 才一起继续，用确定的阶段关系制造交错。
    # gather 接收两个 coroutine，把它们并发调度，并按传入顺序收集返回的二元组；
```

```python
async def demonstrate_stream_and_context() -> None:
    # Barrier(2) 要求两个参与者都到达 wait 才一起继续，用确定的阶段关系制造交错。
    # gather 接收两个 coroutine，把它们并发调度，并按传入顺序收集返回的二元组；
    # 返回顺序不表示 req-A 一定先运行。
    barrier = asyncio.Barrier(2)
    contexts: tuple[tuple[str, str], tuple[str, str]] = await asyncio.gather(
        observe_request_context("req-A", barrier),
        observe_request_context("req-B", barrier),
    )

    # 每个子 Task 在屏障前后都读到自己的 request id；它们的 finally 用 token 恢复上下文。
    # 父 Task 从未设置该 ContextVar，所以 get() 仍得到 agent_runtime 中声明的默认值 "-"。
    print("两个并发请求在屏障前后的值:", contexts)
    print("父上下文恢复为默认值:", request_id_var.get())
    assert contexts == [("req-A", "req-A"), ("req-B", "req-B")]


# 这里比较两种离线 I/O 边界：MockTransport 仍走 HTTPX 的请求/响应处理，
# 临时文件则真的执行 SQLite SQL；两者都不会访问生产服务或长期数据。
async def demonstrate_http_and_database() -> None:
    print_section("6. I/O 分层：MockTransport 与临时 SQLite 文件")

    # MockTransport 会把 AsyncClient 构造好的 Request 直接交给这个同步 handler，
    # 不经过 DNS、socket、TLS 或真实网络。params 中的 query 可从 request.url 读回，
    # 再组装成状态码 200、JSON 对象为响应体的内存 Response。
    def handler(request: httpx.Request) -> httpx.Response:
        return httpx.Response(
            200,
            json={"answer": f"离线回答：{request.url.params['query']}"},
        )

    # async with 表示本函数拥有 client：进入时取得资源，离开时无论成功或异常都异步关闭。
    # base_url 只负责把 fetch_model_reply 使用的相对路径 /model 补成完整 URL；
    # transport 已被显式替换，因此这个看似真实的域名不会产生网络连接。
    async with httpx.AsyncClient(
        transport=httpx.MockTransport(handler),
```

```python
async def demonstrate_http_and_database() -> None:
    async with httpx.AsyncClient(
        transport=httpx.MockTransport(handler),
        base_url="https://model.test",
    ) as client:
        # 原始 dict 先被 SearchArgs 校验，再用于构造 query 参数；handler 的 JSON 响应
        # 随后又被校验成 ModelReply。reply 因而是模型实例，可通过属性访问 answer。
        reply: ModelReply = await fetch_model_reply(client, {"query": "pytest", "limit": 1})
    print("进程内 HTTP transport 返回:", reply.answer)

    # TemporaryDirectory 的 with 作用域拥有整个临时目录：directory 是目录路径字符串，
    # Path(directory) / "notes.sqlite3" 用 / 运算符拼出跨平台路径对象。
    with tempfile.TemporaryDirectory(prefix="chapter27-") as directory:
        database_path: Path = Path(directory) / "notes.sqlite3"
        # initialize_notes 和 read_note 会分别打开真实 SQLite 连接，执行建表、插入和参数化查询；
        # 它比伪造 Connection 更能发现 SQL/绑定错误，但不代表验证了生产数据库驱动。
        initialize_notes(database_path)
        print("临时数据库查询:", read_note(database_path, 2))

    # 离开 with 后 TemporaryDirectory 已递归删除目录，所以路径对象仍存在，文件却不存在。
    # database_path 在 Python 中没有块级作用域，因而这里仍可用它检查清理结果。
    print("临时目录退出后数据库仍存在:", database_path.exists())


# 这段故意调用错误实现，说明一条回归测试怎样观察资源泄漏；示例本身随后负责清理，
# 因此完整运行 example.py 不会把未关闭的异步生成器遗留给事件循环。
async def demonstrate_controlled_defect() -> None:
    print_section("7. 受控缺陷：测试应能看到提前返回时流仍未关闭")
    state = StreamState()

    # buggy_read_first_token 用 anext() 取得第一个值后直接返回二元组：first 是 "agent"，
    # leaked_stream 是仍暂停在 yield 处的异步生成器对象，其 finally 还没有执行。
    first: str, leaked_stream: AsyncGenerator[str, None] = await buggy_read_first_token(state)
    try:
        # 此刻 closed 必为 False；正式受控失败测试会断言它应为 True，并准确地变红。
        # 这里仅打印预期失败点，让普通演示仍能继续运行到底。
```

```python

# 这段故意调用错误实现，说明一条回归测试怎样观察资源泄漏；示例本身随后负责清理，
# 因此完整运行 example.py 不会把未关闭的异步生成器遗留给事件循环。
async def demonstrate_controlled_defect() -> None:
    print_section("7. 受控缺陷：测试应能看到提前返回时流仍未关闭")
    state = StreamState()

    # buggy_read_first_token 用 anext() 取得第一个值后直接返回二元组：first 是 "agent"，
    # leaked_stream 是仍暂停在 yield 处的异步生成器对象，其 finally 还没有执行。
    first: str, leaked_stream: AsyncGenerator[str, None] = await buggy_read_first_token(state)
    try:
        # 此刻 closed 必为 False；正式受控失败测试会断言它应为 True，并准确地变红。
        # 这里仅打印预期失败点，让普通演示仍能继续运行到底。
        print("缺陷函数返回:", first, "；此刻 closed:", state.closed.is_set())
        print("回归断言 state.closed.is_set() 会准确失败")
    finally:
        # 演示完立即清理，入口不会遗留未关闭的异步生成器。
        # aclose() 会让生成器执行其 finally；await 保证清理完成后才离开本函数。
        await leaked_stream.aclose()
    print("受控清理后 closed:", state.closed.is_set())


# 统一异步入口按顺序运行七段演示。每个 await 都等当前段完整结束后再进入下一段，
# 所以输出易于阅读；并发只发生在各演示明确创建 Task/gather 的内部。
async def run_examples() -> None:
    # 普通 def 可直接调用；它会在当前线程同步执行完再返回。
    demonstrate_sync_validation()
    await demonstrate_cancellation_cleanup()
    await demonstrate_partial_and_total_failure()
    await demonstrate_concurrency_limit()
    await demonstrate_stream_and_context()
    await demonstrate_http_and_database()
    await demonstrate_controlled_defect()


# main 是同步世界与异步世界的唯一边界。asyncio.run() 创建事件循环、运行 coroutine，
```

```
348  async def run_examples() -> None:
355      await demonstrate_http_and_database()
356      await demonstrate_controlled_defect()
357
358
359  # main 是同步世界与异步世界的唯一边界。asyncio.run() 创建事件循环、运行 coroutine,
360  # 并在结束时关闭异步生成器和循环；已有运行中事件循环的 async 函数里不能再次调用它。
361  def main() -> None:
362      asyncio.run(run_examples())
363
364
365  # Python 导入模块时 __name__ 是模块名，不执行 main；直接运行此文件时它才等于
366  # "__main__"。这与 Java 的固定 main 方法入口不同，是 Python 常见的可导入/可运行双用途写法。
367  if __name__ == "__main__":
368      main()
369
```



example.py .../14_data_models | example.py .../15_protocols_generics | example.py .../16_decorators | pytest.ini

java_to_python > chapters > 27_pytest_async > pytest.ini

```ini
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
addopts = -ra --strict-markers
asyncio_mode = strict
asyncio_default_fixture_loop_scope = function
asyncio_default_test_loop_scope = function
markers =
    asyncio: 由 pytest-asyncio 在其事件循环中运行 coroutine 测试

```

~/vscode/python-quickstart-for-java/java_to_python/chapters/27_pytest_async/README.md

```
1  # 第 27 章测试依赖；-r 同时安装示例运行依赖。
2  -r requirements.txt
3  pytest==9.1.1
4  pytest-asyncio==1.4.0
5
```

```
# 第 27 章运行依赖  业务模块与 example.py 使用这两个示例库。
httpx==0.28.1
pydantic==2.13.5

```